

Chapitre 1 : Introduction

TD/TP

Table des matières

1	Travaux Dirigés	1
1.1	Chemins d'accès	2
1.2	Droits d'accès.....	2
1.3	Manipulations d'entrées.....	3
1.4	Questions de logique	3
2	Travaux pratiques	3
2.1	Préparatifs	3
2.1.1	Bien utiliser le shell	3
2.1.2	Bien utiliser le man.....	4
2.1.3	Simplifications de commandes	4
2.1.4	Commande interne ou externe ?.....	5
2.2	Tour du propriétaire.....	5
2.2.1	Utilisateurs et groupes	5
2.2.2	Arborescence de fichiers.....	6
2.2.3	Droits d'accès.....	6
2.2.4	Processus et programmes	6
2.3	Expérimentations sur les liens symboliques.....	7
2.4	Expérimentations sur les droits d'accès	8
2.4.1	Concernant un fichier.....	8
2.4.2	Concernant un répertoire	9
2.5	Création et sauvegarde d'une arborescence projet.....	10

1 Travaux Dirigés

Les exercices ci-dessous approfondissent quelques-unes des notions vues en cours. Un papier et un crayon suffisent pour les réaliser, votre ordinateur n'est d'aucune aide.

1.1 Chemins d'accès

- a) On considère le fichier `vimrc`, situé dans le sous-répertoire `vim` du répertoire `etc` situé sous la racine. Quel est le chemin d'accès absolu à ce fichier ? Quel est son chemin d'accès relatif par rapport à votre répertoire personnel ?
- b) On considère le dossier `projetS1` situé dans le répertoire personnel de l'utilisatrice `Lamia`. Quel est son chemin d'accès absolu ? Quel est son chemin d'accès relatif par rapport à votre répertoire personnel ?
- c) `Lamia` a créé le fichier `rapport.odt` dans son répertoire `projetS1`. Quelle est son chemin d'accès absolu ? Quel est son chemin d'accès relatif par rapport au répertoire personnel de `Lamia` ? Par rapport au répertoire `projetS1` ?
- d) `Lamia` est dans un shell, dans son répertoire personnel. Les chemins d'accès ci-dessous sont-ils corrects ? Si oui, que désignent-ils et peut-on les simplifier ? Si non, quel est le chemin d'accès correct ?

```
./projetS1
../lamia/.
/etc/../../etc/passwd
/../../../../tmp
```

- e) On fait l'hypothèse que l'arborescence de fichiers ne contient aucun lien symbolique. Combien une entrée quelconque de l'arborescence a-t-elle de chemins d'accès absolus ? De chemins d'accès relatifs ?

1.2 Droits d'accès

Listez, dans l'ordre probable où le SE va les vérifier, tous les droits que vous devez avoir pour exécuter les commandes suivantes :

- a) Afficher le fichier `vimrc` :

```
$ cat /etc/vim/vimrc
```
- b) Modifier le fichier `vimrc` et quitter par `:wq` :

```
$ vim /etc/vim/vimrc
```
- c) Copier le fichier `passwd` dans le répertoire `/tmp` :

```
$ cp /etc/passwd /tmp
```
- d) Déplacer le fichier `group` dans le répertoire `/tmp` :

```
$ mv /etc/group /tmp
```

1.3 Manipulations d'entrées

Remplissez le tableau ci-dessous en vous aidant des tableaux de commandes donnés dans le cours. Il faut indiquer, pour chaque action (en ligne), la ou les commandes qui réalise l'action pour chacun des trois types d'entrées listés (en colonne).

Action	Fichier	Répertoire	Lien symbolique
Créer			
Consulter le contenu			
Modifier le contenu			
Modifier les droits			
Modifier les dates			
Supprimer			

1.4 Questions de logique

Répondez aux questions suivantes en vous fondant uniquement sur votre connaissance du rôle des commandes, et justifiez votre réponse.

- a) La commande `pwd` prend-elle un argument ?
- b) La commande `cd` prend-elle plusieurs arguments ?
- c) Quel est le nombre minimal d'argument de la commande `cp` ?
- d) Sous quelles formes la destination d'une copie (`cp`) peut-elle s'exprimer ?
- e) Reprenez les questions c) et d) avec la commande `mv`.

2 Travaux pratiques

Les exercices suivants doivent être réalisés sur votre machine Linux, dans un shell, en lançant l'application Terminal Emulator. Un seul shell suffit pour faire les exercices, mais vous pouvez en lancer plusieurs si vous aimez l'aventure !

Commencez par créer un répertoire spécifique à ce TP pour y ranger votre production, et déplacez-vous-y (le nom donné ci-dessous est indicatif, vous pouvez bien sûr le changer) :

```
$ mkdir -p ISYS/TP1
$ cd ISYS/TP1
```

2.1 Préparatifs

Dans ce TP comme dans les suivants, nous allons passer dans le shell de nombreuses commandes. Il faut savoir le faire efficacement, comme expliqué ci-dessous.

2.1.1 Bien utiliser le shell

Relisez les trucs et astuces listés dans le document Maitriser les outils du cours Environnement de travail, partie Shell, et commencez à les appliquer dans ce TP : saisie semi-automatique, historique des commandes, édition de commande. Ils vous feront rapidement gagner beaucoup de temps.

2.1.2 Bien utiliser le man

Comme avant tout grand voyage, il faut vous munir d'un bon guide et apprendre à bien vous en servir ! Tout au long du semestre, consultez systématiquement le man d'une commande que vous utilisez pour la première fois. Vous y découvrirez des possibilités insoupçonnées, sous la forme d'options ou formes particulières, qui simplifieront grandement votre travail par la suite ! L'investissement initial s'avère toujours rentable. Rappelez-vous également que le man est la seule documentation à laquelle vous aurez accès durant vos examens en salle machine.

Commençons dès maintenant. Consultez le man pour répondre aux questions suivantes :

Manuel

- a) Quelles sont les sections du man d'intérêt pour ce cours ?
- b) Dans un synopsis de commande, que signifie le texte en gras ? le texte souligné ? Les crochets « [» et «] » ? Les points de suspension « ... » ?

Commande ls

- c) Comment lister les entrées cachées ?
- d) Dans quel ordre par défaut la commande liste-t-elle les fichiers ?
- e) Comment lister les fichiers du plus récent au plus ancien (date de modification) ?
- f) Comment lister les fichiers du plus ancien au plus récent (date de modification) ?

Commande cat

- g) La commande admet-elle aucun argument ?
- h) Comment afficher le numéro des lignes des fichiers affichés ?
- i) Comment marquer la fin de chaque ligne par un '\$' ?

Commande cp

- f) Quelles sont les deux principales *formes* de la commande ?
- g) Peut-on changer le nom d'un fichier quand on le copie dans un autre répertoire ?
- h) Peut-on copier plusieurs fichiers en même temps ? Comment est désignée la destination ?

Commande mv

- i) Reprenez les questions ci-dessus et appliquez-les à la commande mv.

2.1.3 Simplifications de commandes

Décrivez ce que fait globalement chacun des enchainements de commandes suivants, et essayez de les simplifier, si cela est possible, en tenant compte des deux observations suivantes :

- Les chemins d'accès (relatifs ou absolus) nous permettent de désigner n'importe quelle entrée de l'arborescence, quel que soit notre répertoire courant.
- La plupart des commandes peuvent opérer sur plusieurs entrées en même temps, spécifiées en argument.

a) Enchainement

```
$ ls
$ cd /tmp
$ ls
$ cd /var/log
$ ls
```

b) Enchainement

```
$ cp CV.txt /tmp
$ cd projetS1
$ cp brouillon.txt /tmp
$ cd /bin
$ cp cat /tmp
```

c) Enchainement

Note : on suppose que le répertoire / tmp ne contient aucun sous-répertoire.

```
$ mv foo /tmp/qux
$ mv bar /tmp
$ mv baz /tmp
```

2.1.4 Commande interne ou externe ?

Il est parfois important de savoir si une commande que l'on exécute est interne ou externe au shell, et, si elle est externe, où son programme se trouve dans l'arborescence. C'est notamment le cas pour les commandes qui, comme `test` (voir chapitre 3), existent en deux versions : l'une interne au shell, l'autre externe. Savoir laquelle est exécutée par le shell permet d'accéder à la bonne version de la documentation pour l'utiliser correctement.

- a) À l'aide de la commande `type`, indiquez si les commandes suivantes sont internes ou externes, et où elles se trouvent si elles sont externes : `cd`, `pwd`, `cat` et... `bash`.
- b) À l'aide de la commande `whereis`, vérifiez si les commandes suivantes ont une version externe : `cd` et `pwd`.

2.2 Tour du propriétaire

Partons à la découverte de votre machine Linux : Qui y habite ? Comment est structurée son arborescence de fichiers ? Quels sont vos droits dans cette arborescence ? Quels processus ont démarré avant votre connexion ? Et bien d'autres questions encore.

2.2.1 Utilisateurs et groupes

Examinons quels utilisateurs et groupes d'utilisateurs sont déclarés sur votre machine.

- a) Consultez d'abord le manuel, section 5, pour connaître la structure des fichiers `passwd` et `group`. Pour chaque fichier, identifiez ce que décrit chaque ligne du fichier, ainsi que la structure d'une ligne : séparateur de champs, signification des différents champs.

- b) Ouvrez le fichier `/etc/passwd` avec l'éditeur de texte de votre choix. Combien d'utilisateurs au total sont déclarés ? Sont-ils tous des utilisateurs humains ? Lesquels ont `bash` comme interpréteur de commandes ?
- c) Ouvrez le fichier `/etc/group` avec l'éditeur de texte de votre choix. Combien de groupes au total sont déclarés ? Votre groupe principal vous inclut-il explicitement ou implicitement ? Combien de groupes ont plus d'un utilisateur explicitement déclaré ?

2.2.2 Arborescence de fichiers

Examinons maintenant votre répertoire personnel ainsi que le reste de l'arborescence.

- a) Chaque répertoire contient deux entrées créées automatiquement par le SE : `"` et `..`. Pourquoi n'apparaissent-elles pas spontanément ? Vérifiez qu'elles sont bien là, dans votre répertoire personnel et dans `/tmp` par exemple, en les faisant apparaître.
- b) La structure de l'arborescence Linux est relativement normalisée. Que contiennent dans votre distribution les répertoires `/bin` et `/usr/bin` ? Même question pour les répertoires `/sbin`, `/tmp` et `/home`.
- c) Où se trouve le programme des commandes `cat`, `ls`, `cp` et `mv` ? Vérifiez-le en listant le contenu de leur du répertoire de résidence de ces commandes (commandes `type` ou `whereis`).

2.2.3 Droits d'accès

Examinons maintenant les droits d'accès que vous avez sur l'arborescence de fichiers.

- a) Reprenez l'exercice 1.2 a) et vérifiez si vous avez les droits permettant d'exécuter la commande, en indiquant à chaque étape à quelle catégorie d'utilisateurs vous appartenez : utilisateur propriétaire, groupe propriétaire ou autres. En conclusion, la commande va-t-elle s'exécuter correctement ? Vérifiez-le.
- b) Même question que précédemment pour l'exercice 1.2 b).
- c) Quels sont vos droits sur le répertoire `/tmp` ? Conclusion.

2.2.4 Processus et programmes

Examinons maintenant les processus qui s'exécutent sur votre machine.

- a) À l'aide de la commande `ps -f`, listez les processus attachés à votre Terminal Emulator. Quand ont-ils été lancés ? Pour le compte de qui s'exécutent-ils ? Quel programme exécutent-ils ? Ont-ils une relation de parenté ? Expliquez vos réponses.
- b) À l'aide de la commande `ps -ef`, lister tous les processus qui s'exécutent en ce moment sur votre machine. Pour le compte de qui s'exécutent-ils ? Quel est le premier processus lancé ? Quel est son PID et quel programme exécute-t-il ?

- c) Retracedez en détail la généalogie reliant votre shell au premier processus lancé sur la machine, en donnant pour chaque processus : son PID, le programme exécuté, la date/heure de début d'exécution, l'utilisateur pour le compte de qui il s'exécute.
- d) Combien de processus au total exécutent le programme bash ? Lancez plusieurs émulateurs de terminal supplémentaires et vérifiez à nouveau votre réponse.
- e) À l'aide de la commande `kill` ci-dessous, envoyez le signal `SIGKILL` à l'un des processus exécutant bash pour votre compte. Qu'observez-vous ? Interprétez en consultant le man de la commande `kill`.

```
$ kill -KILL <PID du processus visé>
```

- f) Essayez maintenant de `kill` le tout premier processus lancé sur votre machine, identifié à la question b). Que se passe-t-il ? Interprétez.

2.3 Expérimentations sur les liens symboliques

Nous allons maintenant faire quelques expériences concernant les droits liens symboliques, pour comprendre leur fonctionnement exact.

Nous commençons par créer un fichier cible quelconque en dehors de notre répertoire personnel et créons ensuite deux liens vers ce fichier cible.

- a) À l'aide de la commande `cp`, copiez le fichier `passwd` dans `/tmp`.
- b) À l'aide de la commande `ln -s`, créez dans le répertoire courant un lien symbolique nommé `passwd-ln-abs` qui cible `/tmp/passwd` de façon absolue. Créez ensuite un second lien symbolique nommé `passwd-ln-rel` qui cible également `/tmp/passwd`, mais de façon relative. Par rapport à quel répertoire le chemin relatif à la cible doit être exprimé ?
- c) Listez le contenu du répertoire courant au format long. Quels sont les deux éléments affichés qui vous permettent d'identifier les liens symboliques ? Quel est l'élément qui vous permet de distinguer un ciblage absolu d'un ciblage relatif ?
- d) À l'aide d'un éditeur de texte, éditez le lien `passwd-ln-abs`, modifiez la première du fichier à votre convenance et quittez en sauvegardant. Editez ensuite le lien `passwd-ln-rel`, puis le fichier `/tmp/passwd` et constatez que les trois ne sont qu'un seul et même fichier.
- e) Créez maintenant dans le répertoire courant un lien symbolique nommé `passwd-ln-ln`, qui cible un des deux précédents liens, de façon absolue ou relative, à votre convenance. Editez le lien `passwd-ln-ln` et constatez que vous atteignez une nouvelle fois `/tmp/passwd`, mais au travers maintenant d'une double indirection.
- f) À l'aide de la commande `rm`, supprimez maintenant le fichier `/tmp/passwd` et listez à nouveau le contenu du répertoire courant. Les trois liens ont-ils été supprimés en

conséquence de la suppression de `/tmp/passwd` ? Quelle est l'indice visible de la disparition de leur cible ?

- g) Essayez d'afficher par `cat` les entrées `passwd-ln-abs` et `passwd-ln-rel`. Quel message obtenez-vous ?

2.4 Expérimentations sur les droits d'accès

Nous allons maintenant faire quelques expériences concernant les droits d'accès, pour comprendre leur fonctionnement exact.

Consultez pour commencer le man de la commande `chmod`, en particulier la description des notations symboliques qu'elle utilise pour modifier les droits :

- Les lettres **u**, **g** et **o** désignent respectivement l'utilisateur propriétaire, le groupe propriétaire et les autres utilisateurs (**others**). L'union de ces trois catégories d'utilisateurs est désignée par **a** (**all**) = **u** + **g** + **o**, raccourci utile pour modifier si besoin les droits des trois catégories en même temps.
- Les opérateurs **+**, **-** et **=** permettent respectivement d'ajouter aux droits existants, retirer des droits existants et fixer de nouveaux droits indépendamment de ceux existants.
- Les lettres **r**, **w** et **x** désignent respectivement le droit de lecture, d'écriture et d'exécution étudiés en cours. Il existe d'autres droits, dont **s**, qui seront étudiés au S2.

Avec ces notations, la commande suivante ajoute le droit **x** à l'utilisateur propriétaire, fixe les droits du groupe propriétaire à **rw** et retire le droit **w** aux autres utilisateurs sur les fichiers `foo`, `bar` et `baz` :

```
$ chmod u+x,g=rw,o-w foo bar baz
```

2.4.1 Concernant un fichier

Expérimentons les droits de lecture et d'écriture d'un fichier.

- a) À l'aide de la commande `touch`, créez un fichier vide nommé `fichier.txt`. Qui sont initialement l'utilisateur propriétaire et le groupe propriétaire du fichier ? Quels sont les droits d'accès par défaut du fichier pour les différentes catégories d'utilisateurs ?

2.4.1.1 Lecture

- b) À l'aide de la commande `chmod`, retirez le droit de lecture au propriétaire, c'est-à-dire vous. Essayez d'afficher le contenu du fichier avec la commande `cat`. Quel message obtenez-vous ?

Remarquez que :

- Les droits qui s'appliquent à vous sont bien ceux de l'utilisateur propriétaire, et non ceux du groupe propriétaire, auquel vous appartenez aussi.
- Vous n'avez pas le droit de lecture alors que le groupe propriétaire et les autres l'ont : les droits de chacune des trois catégories d'utilisateurs sont fixés indépendamment les uns des autres, sans hiérarchie ou inclusion.

- c) À l'aide de la commande `echo` ci-dessous, que vous comprendrez au chapitre 2, ajoutez le message « Bonjour ! » à la fin du fichier, puis rétablissez les droits de lecture pour le propriétaire et enfin affichez le contenu du fichier. Que constatez-vous ?

```
$ echo 'Bonjour !' >> fichier.txt
```

Remarquez que, si les droits sont configurés comme tels, vous pouvez écrire dans un fichier sans avoir le droit de le lire : lecture et écriture sont deux opérations indépendantes.

2.4.1.2 Ecriture

- d) Retirez maintenant le droit d'écriture au propriétaire, c'est-à-dire vous. Essayez à nouveau d'ajouter le message « Bonjour ! » à la fin du fichier. Quel message obtenez-vous ? Vérifiez, en l'affichant, que le contenu du fichier n'a pas été modifié.
- e) À l'aide de la commande `rm`, tentez de supprimer le fichier. Que constatez-vous ? Répondez « y » à la question posée par `rm`. Le fichier est-il finalement supprimé ? L'absence du droit d'écriture sur le fichier empêche-t-il sa suppression ?

2.4.2 Concernant un répertoire

Expérimentons maintenant les droits de lecture, d'écriture et de traversée d'un répertoire.

- a) À l'aide de la commande `mkdir`, créez dans votre répertoire courant un sous-répertoire vide nommé `repertoire`. Qui sont initialement l'utilisateur propriétaire et le groupe propriétaire du répertoire ? Quels sont les droits d'accès par défaut du répertoire pour les différentes catégories d'utilisateurs ? Sont-ils les mêmes que pour un fichier ?

```
$ mkdir repertoire
```

- b) **Sans quitter** le répertoire courant, à l'aide de la commande `echo` ci-dessous, créez sous `repertoire` un fichier nommé `fichier.txt`, contenant le message « Bonjour ! ». Pouvez-vous lire le contenu de `repertoire` ? Pouvez-vous lire le contenu de `fichier.txt` ?

```
$ echo 'Bonjour !' >> repertoire/fichier.txt
```

```
$ ls -l repertoire
```

```
...
```

```
$ cat repertoire/fichier.txt
```

```
...
```

```
$
```

2.4.2.1 Lecture

- c) À l'aide de la commande `chmod`, retirez le droit de lecture de `repertoire` à son propriétaire, c'est-à-dire vous. Pouvez-vous continuer à lire le contenu de `repertoire` ? À utiliser la saisie semi-automatique pour saisir `fichier.txt` ? À lire le contenu de `fichier.txt` ?

2.4.2.2 Traversée

- d) À l'aide de la commande `chmod`, redonnez le droit de lecture de `repertoire` à son propriétaire et retirez-lui le droit de traversée. Pouvez-vous reprendre la lecture du contenu de `repertoire` ? Utiliser la saisie semi-automatique pour saisir `fichier.txt` ? Lire le contenu de `fichier.txt` ?

2.4.2.3 Ecriture

- e) À l'aide de la commande `chmod`, redonnez le droit de traversée de repertoire à son propriétaire et retirez-lui le droit d'écriture. Pouvez-vous déplacer ou renommer fichier.txt ? Le supprimer de repertoire ? Pouvez-vous copier dans repertoire le fichier /etc/passwd ?

2.5 Création et sauvegarde d'une arborescence projet

On considère le projet Stars, dont l'arborescence de développement est donnée ci-dessous par la commande `tree` :

```
$ tree Stars
Stars
├── bin
│   └── program
├── CHANGELOG → doc/CHANGELOG.md
├── doc
│   ├── CHANGELOG.md
│   ├── README.md
│   └── specifications.md
├── README → doc/README.md
├── src
│   ├── main
│   │   └── main.c
│   └── utils
│       ├── math.c
│       ├── math.h
│       ├── misc.c
│       └── misc.h
└── test
    └── utils
        ├── math.c → ../../src/utils/math.c
        ├── math.h → ../../src/utils/math.h
        └── test-math.c

8 directories, 14 files
```

- a) À l'aide des commandes vues précédemment, reproduisez cette arborescence à l'identique, répertoire Stars inclus. Notez que tous les fichiers sont vides et que la cible des liens symbolique est désignée de façon relative.

La création de cette arborescence ne nécessite que quatre commandes bien pensées, mais vous pouvez utiliser autant de commandes que vous le souhaitez.

La suite utilise la commande `tar`, qui permet de créer, consulter et extraire une archive, éventuellement compressée. Consultez le man pour découvrir la syntaxe et les options qui permettent de réaliser ces opérations.

- b) À l'aide de la commande `tar`, créez une archive compressée au format gzip de cette arborescence, répertoire Stars inclus, archive nommée Stars.tgz.
- c) Consultez le contenu de l'archive générée, et décompressez-la dans le répertoire /tmp pour vous assurer qu'elle est correcte, notamment s'agissant des liens symboliques.
- d) Déposez dans l'espace Moodle TP1 l'archive compressée Stars.tgz.